UNIVERSITY OF INFORMATION TECHNOLOGY & SCIENCES

Lab Manual

CSE 312/CSE0613312

Artificial Intelligence Lab

University of Information Technology and Sciences

Primitives

Department offering the course: Department of Computer Science and Engineering

Entity offering the program: University of Information Technology and Sciences (UITS)

Course Title: Artificial Intelligence Lab

Course Code: CSE 312/CSE0613312

Academic Level/Term: 3 / 1

Credit Hour: 1

Contact Hour per Week: 2

Facilities Required for Teaching and Learning:

- High-performance CPUs and GPUs.
- White Board
- Sufficient Equipment

Server Needed (If any): No

Additional Requirements (if any): As the course emphasizes on online assignment, a total of 25 functional PCs should be kept available.

Assessment

Term Assessment:

Name of the Term Assignment: The variety of assignments might be quite diverse.

Here few conventional project categories are listed:

- Searching algorithms by using python
- Depth-first Search
- Breadth-first Search
- Best First Search Algorithm(Greedy search)
- A* Search Algorithm
- Mini max Algorithm
- Alpha beta pruning
- Fuzzy Logic System

Basic Requirement of the Term Assignment: Basic understanding of Searching algorithms.

Total Duration of the Assignment: 14 weeks.

Presentation Requirement: Yes.

Demonstration Requirement: Depending on the term project, demonstration may be conducted by respective advisors.

Reporting Requirement: As per advisor's decision.

Mark Distribution of a Particular Deliverable: No.

Schedule of the Assignment: Each student should present his/her progress to his/her supervisor in every one/two-week interval.

Final Viva:

Viva should be conducted by the respective project advisor (not any panel of teachers).

Quiz:

Necessity of conducting quiz: Quiz needs to be conducted based on the modular programming paradigms. Quiz must not include specific project-oriented questions.

Type of questions: MCQ, True/False, Small code snippet (like, possible outputs, any existing bug etc.)

Weights of Assessments

Assessments (Based on UCAM)	**Percentage**	**Breakdown Marks**
Class Attendance	20 %	Class Attendance (10)
		Class Performance (10)
Lab Performance	30 %	Lab Test 1 (10)
		Lab Test 2 (20)
Quiz	30%	Quiz (20)
		Lab Report (10)
Viva	20%	Project Presentation (10)
		Viva (10)
Total	**100%**	**100**

Mark Distribution of Class Attendance

Range of Percentage	Marks
90% - 100%	10
80% - 89%	9
70% - 79%	7-8
60% - 69%	5-6
Below 50%	1-4
Below 20%	0

Rubric for Assessment of Lab Performance

Criteria	Levels		
	Novice (0-3)	**Accomplished (4-7)**	**Expert (8-10)**
A. Laboratory Skills			
1. Manipulative Skills	Members do not demonstrate needed skills	Members occasionally demonstrate needed skills	Members always demonstrate needed skills
2. Experimental Set-up	Members are unable to set up the materials.	Members are able to set up the materials with supervision.	Members are able to set up the materials with minimum supervision
3. Process skills	Members do not demonstrate targeted process skills	Members occasionally demonstrate targeted process skills	Members always demonstrate targeted process skills
4. Oral Communication	Members are able to answer questions with many errors	Members are able to correctly answer with few errors	Members are able to correctly answer all questions
5. Safety Precautions	Minimum do not follow safety precautions	Members follow safety precautions most of the times	Members follow safety precautions at all times
B. Work Habits			
6. Time Management	Members do not finish on time with incomplete data	Members finish on time with incomplete data	Members finish Ahead of time with complete data
7. Preparedness	Members do not bring the required materials	Members bring only some of the required materials	Members bring all the required materials
8. Teamwork	Members do not know its task and have no	Members are on task and have defined responsibilities most of time.	Members are on task and have defined responsibilities all the times

	defined responsibilities		
9. Cleanliness	Messy workplace during and after the experiment	Clean and orderly workplace with occasional mess during and after the experiment	Clean and orderly workplace at all times during and after the experiment
10. Ability to work independently	Members require supervision by the teacher at all times	Members require occasional supervision by the teacher	Members do not need to be supervised by the teacher

Rubric for Assessment of Lab Experiment

Parameter	**LOW (1)**	**MEDIUM (2-3)**	**HIGH (4-5)**
Background Knowledge	Sufficient study and half of the questions have been answered correctly.	Adequate study and more than half of the questions have been answered correctly.	Thorough study and all the questions have been answered correctly.
Select Appropriate Equipment & Tools	Satisfactory selection of equipment's and tools to achieve desired objective without proper reasoning.	Satisfactory selection of equipments and tools to achieve desired objective along with proper reasoning.	Relevant and smart selection of equipments and tools to achieve desired objective along with proper reasoning.
Design	Inappropriate Design and contains few technical errors.	Design documentation is appropriately detailed and structured for the intended Purpose with less information	Design documentation is appropriately detailed and structured for the intended Purpose.
Logic Construction	Significant findings are summarized. Acceptable conclusion. Acceptable suggestion for further research.	Significant findings are summarized. Good conclusion. Good suggestion for further research.	Significant findings are summarized. Precisely concluded. Excellent suggestion for further research.
Documentation	The requirements of document writing are	Document meets all prescribed	Document meets all requirements

	not properly addressed.	requirements.	and it is prepared in original and creative way to engage readers.

Rubric for Lab Participation

Proficient (5)	**Adequate (4-3)**	**Substandard (2)**	**Unacceptable (1)**
Student demonstrates an accurate understanding of the lab objectives and concepts. The student can correctly answer questions and if appropriate, can explain concepts to fellow classmates. Student is eager to participate and assists when needed.	Student arrives on time to lab, but may be unprepared. Answers to questions are basic and superficial suggesting that concepts are not fully grasped.	Student tardiness or unpreparedness makes it impossible to fully participate. If able to participate, student has difficulty explaining key lab concepts.	Student was absent from lab or did not participate. There was no attempt to make prior arrangements to make up the lab.

Rubric for Lab Report

Excellent (9-10)	**Good (6-8)**	**Average (3-5)**	**Poor (1-2)**
Student demonstrates an accurate understanding of the lab objectives and concepts. Questions are answered completely and correctly. Graphs are neat, creative and include complete titles and accurate units. Errors, if any are minimal	Student has a basic knowledge of content, but may lack some understanding of some concepts. Questions are answered fairly well and/or graphs could have been done more neatly, accurately or with more complete information.	Student has problems with both the graphs and the answers. Student appears to have not fully grasped the lab content and the graph(s) possess multiple errors.	Student turns in lab report late or the report is so incomplete and/or so inaccurate that it is unacceptable.

Rubric for Lab Viva

Poor (1-2)	Average (3-5)	Good (6-8)	Excellent (9-10)
Tells 1 or 2 events or key facts	Tells some of the events or key Facts	Tells many events, in sequence for the most part, or tells many key facts	Tells most events in sequence or tells most key facts
Includes few or no important details from topics	Includes some important details no Important details from topics	Includes many important details from topics	Includes most important details and key language or vocabulary from topics
Refers to 1 or 2 characters or topics using pronouns (he, she, it, they)	Refers to 1 or 2 characters or topics by generic name or label	Refers to many characters or topics	Refers to all characters or topics by specific name
Responds with incorrect information	Responds with some misinterpretation	Responds with literal interpretation	Responds with interpretation that shows higher level thinking
Provides limited or no response to teacher questions and prompts	Provides some response to teacher questions and prompts	Provides adequate response to teacher questions and prompts	Provides insightful response to teacher questions or prompts

Rubric for Lab Final

Excellent (9-10)	The student completed assigned tasks without any help from the instructor and showed the results appropriately.
Good (6-8)	The student completed assigned tasks with minimal help from the instructor and showed the results appropriately.
Average (3-5)	The student could not complete all assigned tasks and showed partial results
Poor (1-2)	The student did not complete assigned tasks.

Reference Material

Essential Books/Papers:

1. Artificial Intelligence: A Modern Approach (3rd Edition) – Stuart Jonathan Russell, Peter Norvig; Prentice Hall (2010)

Recommended Reference books:

1. Artificial Intelligence: A New synthesis – Nils J. Nilsson; Routledge

Website (if any): https://www.w3schools.com/python/

Handout (if any): NONE

Others: NONE

Index

SL.	Contents	Page No.
1.	Vision of the CSE Department	11
2.	Mission of the CSE Department	11
3.	Program Educational Objectives (PEOs)	11
4.	Course Objectives	11
5.	Course Outcomes (COs)	12
6.	Program Outcomes (POs)	12
7.	Mapping of the Course with COs and POs	13
8.	Introduction to Artificial Intelligence	13

Experiments

Lab Class	Contents	Task	Page No.
1.	Introduction and Overview, Discussion and distinguish the notions of rational behavior and intelligent agents; Basic of Python, Environment setup, Comments, Variable, Data type	Experiment	14
2.	Basic knowledge of strings, numbers, operators, List, tuples, dictionaries	Experiment	23
3.	Basic knowledge of Conditions, Loops, and functions	Experiment	29
4.	Problem solving by searching 4-queen problem	Experiment	33
5.	Implementation of Uninformed Search Algorithms - Depth-first Search	Experiment	37
6.	Implementation of Uninformed Search Algorithms - Breadth-first Search	Experiment	41
7.	Implementation of Informed Search Algorithms - Best First Search Algorithm(Greedy search)	Experiment	44
8.	Implementation of Informed Search Algorithms - A* Search Algorithm	Experiment	48
9.	Implementation of Adversarial Search - Minimax Algorithm	Experiment	54
10.	Implementation of Adversarial Search - Alpha beta pruning	Experiment	62
11.	Implementation of Uninformed Search - Depth-limited Search Algorithm	Experiment	66
12.	Implementation of Uninformed Search -Uniform cost search Algorithm	Experiment	70
13.	. Implement 8-Queens Problem using Python.	Experiment	74
	Complex Engineering Problems	Test	78

Do’s & Don'ts in Laboratory

Do’s

1. Students should be punctual and regular to the laboratory.

2. Students should come to the lab in-time with proper dress code.

3. Students should maintain discipline all the time and obey the instructions.

4. Students should carry observation and record completed in all aspects.

5. Students should be at their concerned experiment table; unnecessary moments are restricted.

6. Students should follow the indent procedure to receive and deposit the components from the lab technician.

7. While doing the experiments any failure/malfunction must be reported to the faculty.

8. Students should check the connections of the circuit properly before switching ON the power supply.

9. Students should verify the reading with the help of the lab instructor after completion of the experiment.

10. Students must ensure that all switches are in the lab OFF position, all the connections are removed.

11. At the end of practical class the apparatus should be returned to the lab technician and

take back the indent slip.

12. After completing your lab session SHUTDOWN the systems, TURNOFF the power switches and arrange the chairs properly.

13. Each experiment should be written in the record note book only after getting a signature from the lab in charge in the observation notebook.

Don’ts

1. Don’t eat and drink in the laboratory.

2. Don’t touch electric wires.

3. Don’t turn ON the circuit unless it is completed.

4. Avoid making loose connections.

5. Don’t leave the lab without permission.

6. Don’t bring mobiles into laboratory.

7. Do not open any irrelevant sites on computer.

8. Don’t use a flash drive on computers

1. Vision of CSE Department:

To scaffold the next generation of engineers and scientists in a student-centered learning environment to reach digital fluency in Computer Science & Engineering. The challenge is to make the students competent and skilled leaders in the wake of the ever-changing and challenging global work environment of the 21st century.

2. Mission of CSE Department:

- To provide outstanding education, training and research to the students with a view to making them good human beings as well.
- To build skilled engineers who can efficiently serve the society and nation as a whole.
- To enhance research activities to global level and also maintain industrial collaboration.

3. Program Educational Objectives (PEOs)

PEO1: Establish themselves as competent professionals/entrepreneurs in the field of Computer Science and Engineering and engage themselves in learning and solving challenging problems in this evolving computational field.

PEO2: Continue to learn and apply multidisciplinary approaches to solving complex engineering problems and participate in multidisciplinary research and innovation.

PEO3: Engage themselves in a self-learning manner which will make them independent at the time of learning and helps to enhance their professional/entrepreneurial careers.

PEO4: Contribute substantially to the society in sustainable upliftment of quality of life through the application of professional/entrepreneurial skills and expertise in a manner that conforms to the ethical and sustainability standards.

4. Course Objectives:

Explain the basic knowledge representation, problem solving, and learning methods of Artificial Intelligence
Assess the applicability, strengths, and weaknesses of the basic knowledge representation, problem solving, and learning methods in solving particular particular engineering problems
Develop intelligent systems by assembling solutions to concrete computational problems

Understand the role of knowledge representation, problem solving, and learning in intelligent system engineering.

5. Course Outcomes (COs):

CLO1	**Understand** the concepts of Artificial intelligence, Intelligent Agents And issues in the design of search programs.
CLO2	**Analyze** and simulate various searching techniques, constraint satisfaction problem and example problems
CLO3	**Analyze** and **design** a real world problem for implementation and understand the dynamic behavior of a system

6. Program Outcomes (POs)

PO_1 Engineering knowledge: Apply knowledge of mathematics, natural science and engineering fundamentals.
PO_2 Problem analysis: Identify, formulate, research literature and analyze complex engineering problems searching substantiated conclusions.
PO_3 Design/development of solutions: Design solutions for complex engineering problems and design systems, components or processes that meet specified needs with appropriate consideration.
PO_4 Investigation: Conduct investigations of complex problems using research-based knowledge and research methods.
PO_5 Modern tool usage: Create, select and apply appropriate techniques, resources, and modern engineering and IT tools.
PO_6 The engineer and society: Apply reasoning informed by contextual knowledge Modern tool usage: Create, select and apply appropriate techniques, resources, and modern engineering and IT tools. to assess societal, health, safety, legal and cultural issues.
PO_7 Environment and sustainability: Understand and evaluate the sustainability and impact of professional engineering work in the solution of complex engineering problems.
PO_8 Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
PO_9 Individual work and teamwork: Function effectively as an individual, and as a member or leader in diverse teams and in multi-disciplinary settings.
PO_10 Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO_11 Project management and finance: Demonstrate knowledge and understanding of engineering management principles, economic decision-making and apply these to multidisciplinary environments.
PO_12 Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

7. Mapping of the Course with COs and POs:

Course Learning Outcomes (CLO) (3-5)	PLO1	PLO2	PLO3	PLO4	PLO5	PLO6	PLO7	PLO8	PLO9	PLO 10	PLO 11	PLO 12
CLO1	√											
CLO2		√										
CLO3											√	

8. Introduction of Artificial Intelligence?

Artificial intelligence, or AI, refers to the simulation of human intelligence by software-coded heuristics. Nowadays this code is prevalent in everything from cloud-based, enterprise applications to consumer apps and even embedded firmware.

The year 2022 brought AI into the mainstream through widespread familiarity with applications of Generative Pre-Training Transformer. The most popular applications are OpenAI's DALL-E text-to-image tool and ChatGPT. The widespread fascination with ChatGPT made it synonymous with AI in the minds of most consumers. However, it represents only a small portion of the ways that AI technology is being used today.

The ideal characteristic of artificial intelligence is its ability to rationalize and take actions that have the best chance of achieving a specific goal. A subset of artificial intelligence is machine learning (ML), which refers to the concept that computer programs can automatically learn

from and adapt to new data without being assisted by humans. Deep learning techniques enable this automatic learning through the absorption of huge amounts of unstructured data such as text, images, or video.

Experiment No. 01:

Experiment Name / Title: Introduction and Overview of Python, Environment setup, Comments, Variable and Data type

Aim or Objectives

1. To learn about the setup of the Python environment .
2. To learn the basic concepts of comment, Variable and Data type of python.

Apparatus Required

- High-performance CPUs and GPUs.

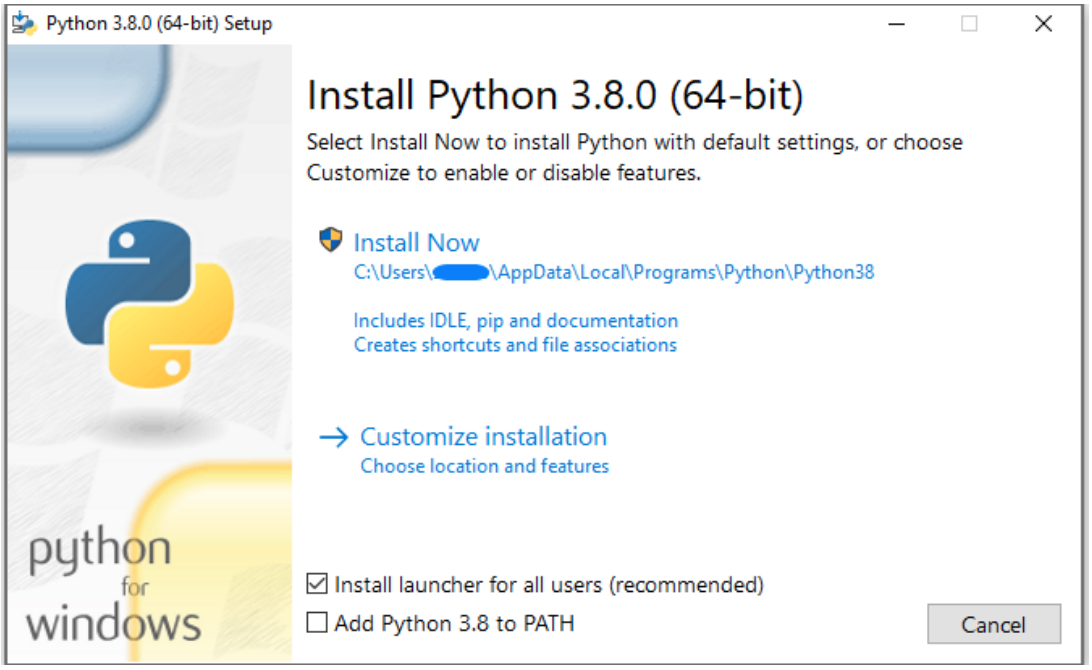
Theory

Before we start Python programming, we need to have an interpreter to interpret and run our programs. For windows the Installation process:

Installation:

There are many interpreters available freely to run Python scripts like IDLE (Integrated Development Environment) which is installed when you install the python software from http://python.org/downloads/ Steps to be followed and remembered:

Step 1: Select Version of Python to Install.
Step 2: Download Python Executable Installer.
Step 3: Run Executable Installer.
Step 4: Verify Python Was Installed On Windows.
Step 5: Verify Pip Was Installed.
Step 6: Add Python Path to Environment Variables (Optional)



Comments:

Comments can be used to explain Python code. Comments can be used to make the code more readable. Comments can be used to prevent execution when testing code.

Variables:

Variables are nothing but reserved memory locations to store values. This means that when you create a variable you reserve some space in memory. Based on the data type of a variable, the interpreter allocates memory and decides what can be stored in the reserved memory. Therefore, by assigning different data types to variables, you can store integers, decimals or characters in these variables.

Rules for Python variables:

• A variable name must start with a letter or the underscore character

• A variable name cannot start with a number

• A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9,

and _)

• Variable names are case-sensitive (age, Age and AGE are three different variables)

Assigning Values to Variables:

Python variables do not need explicit declaration to reserve memory space. The declaration happens automatically when you assign a value to a variable. The equal sign (=) is used to assign values to variables.

The operand to the left of the = operator is the name of the variable and the operand to the

right of the = operator is the value stored in the variable.

Data types:

The data stored in memory can be of many types. For example, a student roll number is stored as a numeric value and his or her address is stored as alphanumeric characters. Python has various standard data types that are used to define the operations possible on them and the storage method for each of them.

Algorithm

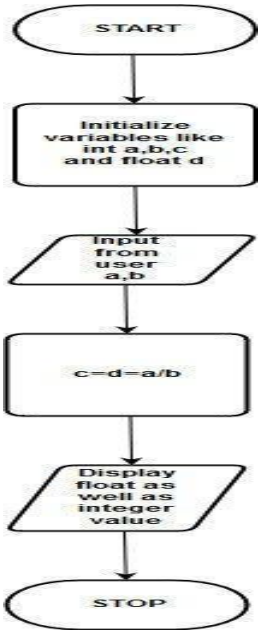
Variable:

1. Choose a name for the variable.
2. Assign a value to the variable using the assignment operator '='.
3. Use valid Python variable naming conventions (e.g., start with a letter or underscore, followed by letters, digits, or underscores).
4. The value assigned can be of any valid Python data type (e.g., integer, float, string).
5. Ensure the variable name is meaningful and reflects the purpose or content of the value it holds.

Data type:

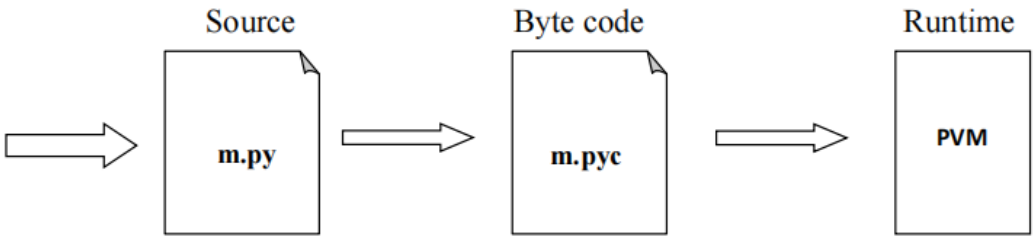
1. Identify the type of data you want to work with.
2. Determine the appropriate Python data type for your data (e.g., integer, float, string, list, tuple, dictionary, etc.).
3. Assign the data to a variable using the appropriate syntax.
4. Perform operations or manipulations on the data as needed using Python's built-in functions and operators.
5. Verify the results and ensure that the data type remains consistent throughout your program.

Flowchart:



Working Procedure

Python's traditional runtime execution model: The source code you type is translated to byte code, which is then run by the Python Virtual Machine (PVM). Your code is automatically compiled, but then it is interpreted.



The source code extension is.py.

The Byte code extension is.pyc (compiled Python code).

There are two modes for using the Python interpreter:

• Interactive Mode

• Script Mode

Code or script

Outcome/ Results (input, output etc.)

Installation:

Without passing a Python script file to the interpreter, directly execute code at the Python prompt.

Once you're inside the python interpreter, then you can start.

```
>>> print("hello world")
hello world
```

Relevant output is displayed on subsequent lines without the >>> symbol

```
>>> x=[0,1,2]
```

Quantities stored in memory are not displayed by default.

```
>>> x
```

#If a quantity is stored in memory, typing its name will display it.

[0, 1, 2]

```
>>> 2+3
5
```

The chevron at the beginning of the 1st line, i.e., the symbol >>> is a prompt the python interpreter uses to indicate that it is ready. If the programmer types 2+6, the interpreter replies 8.

Comments:

Comments starts with a #, and Python will ignore them:

Input:

```
#This is a comment
```

```
print("Hello, World!")
```

Output:

```
Hello, World!
```

Python does not really have a syntax for multiline comments.Or, not quite as intended, you can use a multiline string.Since Python will ignore string literals that are not assigned to a variable, you can add a multiline string (triple quotes) in your code, and place your comment inside it:

Input:

```
"""
This is a comment
written in
more than just one line
"""
print("Hello, World!")
```

Output:

Hello, World!

Variable:

Python variables do not need explicit declaration to reserve memory space. The declaration happens automatically when you assign a value to a variable. The equal sign (=) is used to assign values to variables.

The operand to the left of the = operator is the name of the variable and the operand to the right of the = operator is the value stored in the variable

For example –

```
a= 100 # An integer assignment
b = 1000.0 # A floating point
c = "John" # A string
print (a)
print (b)
print (c)
```

This produces the following result –

```
100
1000.0
John
```

Multiple Assignment:

Python allows you to assign a single value to several variables simultaneously.

For example :

a = b = c = 1

Here, an integer object is created with the value 1, and all three variables are assigned to the same memory location. You can also assign multiple objects to multiple variables.

For example –

a,b,c = 1,2,"mrcet“
Here, two integer objects with values 1 and 2 are assigned to variables a and b respectively, and one string object with the value "john" is assigned to the variable c.

Output Variables:

The Python print statement is often used to output variables.

Variables do not need to be declared with any particular type and can even change type after they have been set.

<u>Data Type:</u>

Int:

Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.

```
>>> print(24656354687654+2)
24656354687656
>>> print(20)
20
>>> print(0b10)
2
>>> print(0B10)
2
>>> print(0X20)
32
>>> 20
20
>>> 0b10
2
>>> a=10
>>> print(a)
10
```

 # To verify the type of any object in Python, use the type() function:

```
>>> type(10)
<class 'int'>
>>> a=11
>>> print(type(a))
<class 'int'>
```

Float:

Float, or "floating point number" is a number, positive or negative, containing one or more decimals.

Float can also be scientific numbers with an "e" to indicate the power of 10.

```
>>> y=2.8
>>> y
2.8
>>> y=2.8
>>> print(type(y))
<class 'float'>
```

```
>>> type(.4)
<class 'float'>
>>> 2.
```

String:

1. Strings in Python are identified as a contiguous set of characters represented in the quotation marks. Python allows for either pairs of single or double quotes.

• 'hello' is the same as "hello".

• Strings can be output to screen using the print function. **For example: print("hello").**

```
>>> print("mrcet college")
mrcet college
>>> type("mrcet college")
<class 'str'>
>>> print('mrcet college')
mrcet college
>>> " "
' '
```

If you want to include either type of quote character within the string, the simplest way is to delimit the string with the other type. If a string is to contain a single quote, delimit it with double quotes and vice versa:

```
>>> print("mrcet is an autonomous (') college")
mrcet is an autonomous (') college
>>> print('mrcet is an autonomous (") college')
mrcet is an autonomous (") college
```

Discussion

Python installation is the initial step towards programming proficiency. Begin by downloading Python from the official website and following the installation instructions. Ensure Python is added to the system's PATH for easy accessibility. Comments writing is a crucial aspect of coding for clarity and documentation. Use comments to explain code functionality, provide context, and make it easier for others to understand your code. Use the '#' symbol to start a comment in Python. Variables in Python are containers used to store data values. They can hold various data types such as integers, floats, strings, lists, tuples, dictionaries, etc. Variables are assigned using the '=' operator. Understanding data types is fundamental in Python programming. Python supports dynamic typing, allowing variables to change data types as needed. Common data types include integers (int), floating-point numbers (float), strings (str), lists (list), tuples (tuple), dictionaries (dict), and Boolean (bool).Learning to install Python, write meaningful comments, declare variables, and understand data types lays a solid foundation for Python programming. Practice these concepts to enhance your coding skills and problem-solving abilities.

Conclusion

By understanding the installation process, students can seamlessly set up Python on their systems, enabling them to delve into coding without hindrance. Comments serve as a roadmap for understanding code, enhancing collaboration and code maintenance. Variables, the building blocks of Python programming, enable the storage and manipulation of data. Familiarity with different data types empowers students to efficiently handle diverse information and tackle programming challenges with confidence. As students embark on their journey into Python programming, embracing these fundamental concepts equips them with the necessary skills to explore advanced topics and tackle real-world problems effectively. Through consistent practice and experimentation, students can cultivate a strong foundation in Python programming, paving the way for innovation and creativity in their coding endeavors.

Viva questions

Python Installation:

1. What steps are involved in installing Python on a Windows/Linux/Mac system?
2. Why is it important to add Python to the system's PATH during installation?
3. Can you explain the difference between installing Python from the official website and using package managers like pip?

Comments Writing:

1. What is the purpose of writing comments in Python code?
2. Can you demonstrate how to write single-line and multi-line comments in Python?
3. How can comments enhance the readability and maintainability of code?

Variables:

1. What is a variable in Python, and how is it different from constants?
2. How do you declare and initialize variables in Python?
3. Is Python statically or dynamically typed? Explain.

Data Types:

1. Name and describe at least five different data types available in Python.
2. How does Python determine the data type of a variable?
3. Can you provide examples of type conversion in Python?

Assignment / Homework/Exercise

Assignment Tasks:

Python Installation:

Install Python on your local machine following the official installation guide provided on the Python website.

Verify the installation by opening a terminal or command prompt and typing python --version. Record the Python version installed on your system.

Comments Writing:

Write a Python script that performs a simple arithmetic operation (e.g., addition, subtraction).

Use comments to describe each step of the arithmetic operation and provide context for your code.

Include both single-line and multi-line comments to demonstrate your understanding of comment syntax.

Variables:

Create a Python script that declares and initializes variables of different data types (e.g., integer, float, string, list, dictionary).

Perform basic operations (e.g., arithmetic operations, concatenation) using these variables.

Print the results of the operations to the console for verification.

Data Types:

Explore Python's dynamic typing feature by assigning different data types to variables and observing the behavior.

Experiment with type conversion functions (e.g., int(), float(), str()) to convert between different data types.

Write comments to explain the purpose and outcome of each experiment.

Submission Guidelines:

- Organize your assignment tasks into a structured folder or directory.
- Include a README file documenting the steps you followed to complete each task.
- Ensure your Python scripts are well-commented and formatted for readability.
- Compress the folder containing your assignment files into a single ZIP file for submission.

Experiment No. 02:

Experiment Name / Title: Basic knowledge of Operators, List, Tuples, and Dictionaries

Aim or Objectives

To learn the basic concepts of Operators, List and Tuples and Dictionaries of python.

Apparatus Required

- High-performance CPUs and GPUs.

Theory

Operators:

Operators are used to perform operations on variables and values.Python divides the operators in the following groups:

1. Arithmetic operators
2. Assignment operators
3. Comparison operators
4. Logical operators
5. Identity operators
6. Membership operators
7. Bitwise operators

List:

Lists are used to store multiple items in a single variable. Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are Tuple, Set, and Dictionary, all with different qualities and usage.

Tuples:

Tuples are used to store multiple items in a single variable. Tuple is one of 4 built-in data types in Python used to store collections of data, the other 3 are List, Set, and Dictionary, all with different qualities and usage. A tuple is a collection which is ordered and unchangeable. Tuples are written with round brackets.

Dictionaries:

Dictionaries are used to store data values in key:value pairs. A dictionary is a collection which is ordered*, changeable and do not allow duplicates.

Algorithm

Operators

Find Maximum of Three Numbers

1. Input: Three integers a, b, and c.

2. Comparison: Use comparison operators to determine the maximum value.

Lists

Reverse a List

1. Input: A list of elements.
2. Processing: Reverse the list using list operations.

Tuples

Swap Two Elements Using Tuples

1. Input: Two variables `x` and `y`.
2. Processing: Swap their values using tuple unpacking.

Dictionaries

Count Occurrences of Characters in a String

1. Input: A string.
2. Processing: Count each character's occurrences and store them in a dictionary.

Code or Script

Operator:

```
print(10 + 5)
print(5 + 4 - 7 + 3)
```

Outcome/ Results (input, output etc.)

```
15
5
```

List:

```
thislist = ["apple", "banana", "cherry"]
print(thislist)
print(len(thislist))
thislist.append("orange")
print(thislist)
```

```
thislist.insert(1, "mango")
print(thislist)
thislist.sort()
print(thislist)
```

Outcome/ Results (input, output etc.)

```
["apple", "banana", "cherry"]
3
['apple', 'banana', 'cherry', 'orange']
['apple', 'banana', 'cherry', 'mango']
['banana', 'kiwi', 'mango', 'orange', 'pineapple']
```

Tuple:

```
thistuple = ("apple", "banana", "cherry")
print(thistuple)
print(len(thistuple))

for x in thistuple:
  print(x)
```

Outcome/ Results (input, output etc.)

('apple', 'banana', 'cherry')

3

apple

banana

cherry

Dictionaries:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict)
thisdict["color"] = "red"
print(thisdict)
myfamily = {
  "child1" : {
```

```
    "name" : "Emil",
    "year" : 2004
  },
  "child2" : {
    "name" : "Tobias",
    "year" : 2007
  },
  "child3" : {
    "name" : "Linus",
    "year" : 2011
  }
}

print(myfamily)
```

Outcome/ Results (input, output etc.)

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'color': 'red'}
{'child1': {'name': 'Emil', 'year': 2004}, 'child2': {'name': 'Tobias', 'year': 2007}, 'child3': {'name': 'Linus', 'year': 2011}}
```

Discussion

The lab experiment focused on reinforcing basic concepts of Operators, Lists, Tuples, and Dictionaries, fundamental components of Python programming. In the discussion, it's important to highlight the significance of each concept. Operators serve as tools for mathematical computations and logical operations, enhancing the capability to manipulate data. They enable programmers to perform tasks like addition, subtraction, comparison, and logical operations efficiently. Lists are versatile data structures used to store collections of items. They offer flexibility in handling data, allowing for easy modification, traversal, and manipulation. Participants likely gained insights into list indexing, slicing, appending, and other operations through practical exercises. Tuples, similar to lists, are sequences of elements, but they are immutable. This key difference was likely emphasized in the experiment, showcasing situations where immutability is advantageous, such as ensuring data integrity or

optimizing performance. Lastly, dictionaries provide a powerful way to store data in key-value pairs, facilitating efficient retrieval and modification. Participants likely explored dictionary manipulation methods and learned about their usefulness in various applications, such as data representation and organization. Overall, through hands-on experimentation, participants would have solidified their understanding of these fundamental concepts, laying a strong foundation for further exploration in Python programming.

Conclusion

In conclusion, this laboratory experiment has provided invaluable insight into the fundamental concepts of programming through the exploration of operators, lists, tuples, and dictionaries. Through hands-on experimentation, we have deepened our understanding of how these essential components function within Python, a versatile and widely-used programming language. Operators serve as the building blocks for manipulating data, allowing us to perform arithmetic, comparison, and logical operations with ease. By exploring their various types and applications, we have gained a deeper appreciation for their role in programming. Lists, tuples, and dictionaries represent powerful data structures that enable efficient storage and manipulation of information. Through this experiment, we have learned how to create, access, and modify elements within these structures, as well as their respective strengths and limitations in different scenarios. Moreover, this experiment has provided practical experience in applying these concepts to solve real-world problems, fostering critical thinking and problem-solving skills essential in the field of programming. In conclusion, by gaining proficiency in operators, lists, tuples, and dictionaries, we have laid a solid foundation for further exploration and mastery of Python programming, empowering us to tackle more complex challenges with confidence and skill.

Viva questions

1. What is the difference between a list and a tuple in Python? How would you choose between using a list and a tuple in your code?
2. Explain the concept of dictionary comprehension in Python. Provide an example of how you would use dictionary comprehension to create a dictionary.
3. Discuss the importance of operators in Python programming. Can you give examples of different types of operators and their use cases?

4. How would you access elements in a nested list or tuple in Python? Provide an example to illustrate your answer.

5. Describe the process of adding, removing, and updating elements in a dictionary in Python. Can you provide a code snippet demonstrating these operations?

Assignment / Homework

- How would you access elements in a nested list or tuple in Python? Provide an example to illustrate your answer.

Lab Report: To learn the basic concepts of Operators, List and Tuples and Dictionaries of python

Exercise

```
a. car ={
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(-------------)

b. fruits = ("apple", "banana", "cherry")

Print(--------------)
```

Experiment No. 03:

Experiment Name / Title: Basic knowledge of Condition, Loop, and Functions.

Aim or Objectives

To learn the basic concepts of Condition, Loop, and Functions. of python.

Apparatus Required

- High-performance CPUs and GPUs.

Theory

Conditions:

Python If ... Else:

Python supports the usual logical conditions from mathematics:

- Equals: a == b
- Not Equals: a != b
- Less than: a < b
- Less than or equal to: a <= b
- Greater than: a > b
- Greater than or equal to: a >= b

These conditions can be used in several ways, most commonly in "if statements" and loops.

An "if statement" is written by using the if keyword.

Loop:
Python For Loops:

A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

This is less like the for keyword in other programming languages, and works more like an iterator method as found in other object-oriented programming languages.

With the for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

Functions:

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function.

A function can return data as a result.

Algorithm

Conditions

1. Input: An integer `n`.
2. Condition: Check if `n` is divisible by 2.
3. Output: Print whether `n` is even or odd.

Loops

1. Input: An integer `N`.
2. Initialize: A variable `sum` to 0.
3. Loop: Iterate from 1 to `N`, adding each number to `sum`.
4. Output: Print the final value of `sum`.

Functions

Fibonacci Sequence

1. Input: An integer `n` representing the position in the Fibonacci sequence.
2. Output: The Fibonacci number at position `n`.

Code or Script

Conditions:

```
a = 33
b = 200
if b > a:
  print("b is greater than a")
```

Outcome/ Results (input, output etc.)

b is greater than a

Loop:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
```

Outcome/ Results (input, output etc.)

apple

banana

Cherry

Function:

```
def my_function(fname):
  print(fname + " Refsnes")

my_function("Emil")
my_function("Tobias")
my_function("Linus")
```

Outcome/ Results (input, output etc.)

Emil Refsnes

Tobias Refsnes

Linus Refsnes

Discussion

In this lab experiment focused on basic concepts of conditions, loops, and functions, students delve into fundamental programming structures essential for any budding programmer. Through hands-on exercises, participants grasp the significance of conditional statements, understanding how they enable the execution of specific code blocks based on predefined criteria. Moreover, they explore the versatility of loops, realizing their efficiency in repeating tasks until certain conditions are met. By dissecting functions, students uncover the power of modular programming, appreciating how breaking down tasks into reusable blocks enhances code readability and maintainability. Through this experiential learning journey, participants not only acquire practical coding skills but also cultivate problem-solving abilities crucial in navigating the intricate landscape of software development. Overall, this lab fosters a solid foundation for students to build upon as they advance in their programming journey.

Conclusion

In conclusion, this lab experiment has provided a foundational understanding of key programming concepts including conditions, loops, and functions. Through hands-on practice and experimentation, participants have gained insight into how these fundamental building blocks operate within code structures. By grasping the logic behind conditional statements, iterative loops, and modular functions, learners are equipped with essential skills for problem-solving and program development. Additionally, the practical exercises offered invaluable opportunities for application and reinforcement of theoretical knowledge. As a result, participants are better prepared to navigate more complex programming challenges with

confidence. Moving forward, continued practice and exploration of these concepts will further solidify comprehension and pave the way for more advanced programming endeavors. This experiment serves as a crucial stepping stone in the journey towards mastering programming fundamentals and lays a strong foundation for future learning and growth in the field of computer science.

Viva questions

1. Can you explain the concept of conditions in programming and how they were utilized in this lab experiment?
2. How did you implement loops in the code, and why were they necessary for achieving the desired functionality?
3. Discuss the role of functions in this experiment. How did you structure your code using functions, and what advantages did this approach offer?
4. Can you describe a scenario where you encountered a logical error related to conditions or loops during the experiment, and how did you debug it?
5. How did you ensure the efficiency and effectiveness of your code in terms of conditionals, loops, and function usage?

Assignment / Homework

- How did you implement loops in the code, and why were they necessary for achieving the desired functionality?

Lab Report: To learn Basic knowledge of Condition, Loop, and Functions of python

Exercise

a.---------- :

```
print("Hello from a function")
```

b. a = 50

b = 10

---a---b---

```
print("Hello World")
```

Experiment No. 04:

Experiment Name / Title: Problem solving by searching 4-queen problem.

Aim or Objectives

3. To learn the concepts of Problem solving by searching 4-queen problem.

Apparatus Required

- High-performance CPUs and GPUs.

Theory

The four queens problem is the problem of placing eight queens on a 4×4 chessboard such that none of them attack one another (no two are in the same row, column, or diagonal). More generally, the n queens problem places n queens on an n×n chessboard. There are different solutions for the problem.

Explanation:

- This **pseudocode uses a backtracking algorithm** to find a solution to the 4 Queen problem, which consists of placing 8 queens on a chessboard in such a way that no two queens threaten each other.
- The algorithm starts by placing a queen on the first column, then it proceeds to the next column and places a queen in the **first safe** row of that column.
- If the algorithm reaches the 8th column and al**l queens are placed in a safe position**, it prints the board and returns true.
  If the algorithm is unable to place a queen in a safe position in a certain column, it backtracks to the previous column and tries a different row.
- The "isSafe" function **checks** if it is safe to place a queen on a certain row and column by checking if there are any queens in the same row, diagonal or anti-diagonal.
- It's worth to notice that this is just a high-level **pseudocode** and it might need to be adapted depending on the specific implementation and language you are using.

Algorithm

1. **Initialize**: Start with an empty 4x4 board and a function to place queens.

2. **Place Queen**: Try placing a queen in each column of the current row.
3. **Check Validity**: Ensure the queen does not conflict with others already placed (no same row, column, or diagonals).
4. **Recursive Call**: Move to the next row and repeat placement.
5. **Backtrack**: If placing a queen leads to a conflict, remove it and try the next column.
6. **Base Case**: Return the board configuration when all 4 queens are placed successfully.

Code or Script

```
#Number of queens
print ("Enter the number of queens")
N = int(input())

#chessboard
#NxN matrix with all elements 0
board = [[0]*N for _ in range(N)]

def is_attack(i, j):
    #checking if there is a queen in row or column
    for k in range(0,N):
        if board[i][k]==1 or board[k][j]==1:
            return True
    #checking diagonals
    for k in range(0,N):
        for l in range(0,N):
            if (k+l==i+j) or (k-l==i-j):
                if board[k][l]==1:
                    return True
    return False

def N_queen(n):
    #if n is 0, solution found
    if n==0:
        return True
    for i in range(0,N):
        for j in range(0,N):
            '''checking if we can place a queen here or not
            queen will not be placed if the place is being
attacked or already occupied'''
            if (not(is_attack(i,j))) and (board[i][j]!=1):
                board[i][j] = 1
                #recursion
                #whether we can put the next queen with this
arrangement or not
                if N_queen(n-1)==True:
                    return True
                board[i][j] = 0
```

```
    return False

N_queen(N)
for i in board:
    print (i)
```

Outcome/ Results (input, output etc.)

```
Enter the number of queens
4
[0, 1, 0, 0]
[0, 0, 0, 1]
[1, 0, 0, 0]
[0, 0, 1, 0]
```

Discussion

In this lab experiment focused on problem-solving by searching, we tackled the classic 4-queen problem, a quintessential challenge in computer science and artificial intelligence. The objective was to place 4 queens on an n×n chessboard in such a way that no two queens threaten each other. Our exploration delved into various search algorithms, evaluating their effectiveness in finding solutions. We analyzed strategies like breadth-first search, depth-first search, and heuristic approaches like hill climbing or simulated annealing. Through this experiment, we gained valuable insights into the strengths and limitations of different search algorithms when applied to combinatorial optimization problems. Furthermore, we deepened our understanding of how algorithmic choices impact efficiency and solution quality. This hands-on experience not only honed our problem-solving skills but also underscored the importance of selecting appropriate algorithms tailored to specific problem domains.

Conclusion

In conclusion, the 4-queen problem serves as an excellent platform for understanding and implementing problem-solving strategies through searching algorithms. Through this lab experiment, we delved into various search algorithms such as depth-first search, breadth-first search, and backtracking. We witnessed how these algorithms can efficiently traverse through the solution space to find valid configurations for placing n queens on an n×n chessboard without threatening each other. Each algorithm exhibited unique characteristics in terms of time complexity, space complexity, and optimality. By analyzing the performance of these algorithms, we gained insights into their strengths and limitations, which can be applied to solve other combinatorial optimization problems. Overall, this experiment provided a valuable

hands-on experience in problem-solving by searching, enhancing our understanding of algorithmic techniques and their practical applications.

Viva questions

1. Can you explain the concept of the 4-queen problem and why it is considered a problem-solving task?
2. What search algorithm did you choose to solve the n-queen problem in this experiment, and what were the reasons behind your choice?
3. Describe the representation of the state space and the search strategy employed in your implementation to find a solution for the 4-queen problem.
4. How did you evaluate the efficiency and effectiveness of your search algorithm in solving the n-queen problem for different values of n?
5. Can you discuss any optimizations or enhancements you implemented to improve the performance of your search algorithm while solving the n-queen problem?

Assignment / Homework

- What search algorithm did you choose to solve the 4-queen problem in this experiment, and what were the reasons behind your choice?

Lab Report: To learn the concepts of Problem solving by searching - 4-queen problem.

Experiment No. 05:

Experiment Name / Title: Implementation of Uninformed Search Algorithms - Depth-first Search

Aim or Objectives

To learn the concepts of Uninformed Search Algorithms - Depth-first Search

Apparatus Required

- High-performance CPUs and GPUs.

Theory

Depth-first Search (DFS) is a fundamental uninformed search algorithm widely used in solving various computational problems, particularly in traversing or searching tree or graph data structures. The theory behind DFS lies in its simplicity and efficiency. The algorithm begins by exploring a single branch of a tree structure as deeply as possible before backtracking. This is achieved by systematically visiting each adjacent node, prioritizing the deepest unexplored node. The process continues until either the goal node is found or all nodes have been visited. DFS operates with a Last-In-First-Out (LIFO) approach, typically implemented using a stack data structure. Its simplicity makes it easy to understand and implement, requiring minimal memory overhead compared to other search algorithms. However, DFS doesn't guarantee optimality in finding the shortest path, and it may get stuck in infinite loops if the graph contains cycles. Understanding the theory behind DFS enables the effective implementation and analysis of its behavior in various problem-solving scenarios.

Algorithm

1. **Initialize**: Start with the open list (stack) containing the initial node and a set for visited nodes.
2. **Expand Node**: Pop the top node from the stack.
3. **Check Goal**: Return the path if the goal node is reached.
4. **Generate Children**: Push all unvisited child nodes onto the stack.
5. **Mark as Visited**: Add the current node to the visited set to avoid revisits.
6. **Repeat**: Continue until the goal is found or the stack is empty.

Code or Script

```
# Sample graph represented as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}#dictionary

visited = set()

def dfs(graph, vertex):
    visited.add(vertex)
    print(vertex)

    for neighbor in graph[vertex]:
        if neighbor not in visited:
            dfs(graph, neighbor)

# Starting DFS from vertex 'A'
dfs(graph, 'A')
```

Outcome/ Results (input, output etc.)

```
A
B
D
E
F
C
```

Discussion

In this lab experiment, we explored the implementation of the Depth-first Search (DFS) algorithm as part of uninformed search strategies. DFS, characterized by its depth-first exploration of a graph or tree, proved to be a fundamental algorithm in traversing and searching through various data structures. Through hands-on implementation, participants gained a deeper understanding of DFS's mechanics, including its stack-based approach and recursive nature. During the experiment, observations were made regarding DFS's effectiveness in searching large, complex graphs, as well as its potential limitations, such as its vulnerability to infinite paths and lack of optimality. By analyzing the algorithm's performance on different datasets, participants could discern its strengths and weaknesses in various scenarios.

Furthermore, discussions delved into DFS's applications in real-world problems, such as maze-solving, pathfinding, and network traversal. Participants explored strategies to mitigate DFS's shortcomings, such as implementing depth limits or incorporating backtracking mechanisms. Overall, this lab provided invaluable insights into the workings of DFS, equipping participants with essential skills in algorithmic problem-solving and search strategy design.

Conclusion

In conclusion, the implementation of the Depth-first Search (DFS) algorithm in this lab experiment has provided valuable insights into its functionality and efficiency within the realm of uninformed search algorithms. Through practical application, we observed DFS's ability to systematically traverse through a search space by exploring as far as possible along each branch before backtracking. This characteristic makes DFS particularly suitable for scenarios where the search space is extensive but shallow. However, it is crucial to note DFS's limitation of potentially getting stuck in infinite loops within environments featuring cyclic paths. Thus, while DFS offers simplicity and effectiveness in certain contexts, its applicability must be carefully considered relative to the problem at hand. This experiment not only reinforced theoretical understanding but also emphasized the significance of algorithmic choices in optimizing search strategies for various real-world applications.

Viva questions

1. Can you explain the basic principles of Depth-first Search (DFS) algorithm and how it differs from other uninformed search algorithms?
2. During the implementation of DFS, what data structure(s) did you use to maintain the frontier and explored nodes? Why did you choose this/these particular data structure(s)?

3. How does DFS handle loops or cycles in the search space? Did you encounter any challenges related to this during the implementation?

4. In terms of time and space complexity, what are the advantages and disadvantages of DFS compared to other uninformed search algorithms such as Breadth-first Search (BFS)?

5. Can you discuss any scenarios or types of problems where DFS might be more suitable or effective compared to other search algorithms?

Assignment / Homework

- Discuss any scenarios or types of problems where DFS might be more suitable or effective compared to other search algorithms?

Lab Report: To learn the concepts of Uninformed Search Algorithms - Depth-first Search.

Experiment No. 06:

Experiment Name / Title: Implementation of Uninformed Search Algorithms - Breadth-first Search

Aim or Objectives

To learn the concepts of Uninformed Search Algorithms - Breadth-first Search

Apparatus Required

- High-performance CPUs and GPUs.

Theory

Breadth-first search (BFS) is a fundamental uninformed search algorithm used to explore and traverse a graph or tree systematically. It operates by expanding nodes in all possible directions at each level of the search tree, prioritizing nodes closer to the root before moving to deeper levels. BFS guarantees the shortest path to the goal node in terms of the number of edges traversed, making it particularly useful for finding solutions in unweighted graphs or when the shortest path is required. In this lab experiment, the implementation of BFS will be explored to understand its mechanics and performance characteristics. The algorithm's time complexity is O(V + E), where V is the number of vertices and E is the number of edges. Through this experiment, students will gain insights into how BFS efficiently explores graphs and its applications in various problem-solving scenarios. Additionally, they will analyze its advantages, limitations, and practical implications.

Algorithm

1. **Initialize**: Start with the open list (queue) containing the initial node and a set for visited nodes.
2. **Expand Node**: Dequeue the front node from the queue.
3. **Check Goal**: Return the path if the goal node is reached.
4. **Generate Children**: Enqueue all unvisited child nodes to the queue.
5. **Mark as Visited**: Add the current node to the visited set to avoid revisits.
6. **Repeat**: Continue until the goal is found or the queue is empty.

Code or Script

```
# Define a graph as a dictionary of nodes and their adjacent nodes
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
```

```
    'D': [],
    'E': ['F'],
    'F': []
}

def bfs(graph, start_node):
    # Initialize the visited set and the queue
    visited = set()
    queue = deque([start_node])

    # Traverse the graph using BFS
    while queue:
        # Dequeue a node from the queue and mark it as visited
        node = queue.popleft()
        visited.add(node)
        print(node, end=' ')

        # Enqueue all adjacent nodes that haven't been visited
        for neighbor in graph[node]:
            if neighbor not in visited:
                queue.append(neighbor)

# Call the BFS function with the graph and the starting node
bfs(graph, 'A')
```

Outcome/ Results (input, output etc.)

A B C D E F

Discussion

The implementation of the Breadth-first Search (BFS) algorithm in this lab experiment offers valuable insights into uninformed search strategies. BFS is renowned for its simplicity and effectiveness in finding the shortest path in unweighted graphs. Through this experiment, we explored BFS's fundamental principles, such as expanding nodes level by level, maintaining a queue for traversal, and marking visited nodes to avoid revisiting them. During the experiment, we observed BFS's ability to systematically explore all possible paths from the start node before moving on to deeper levels. This characteristic ensures optimality in finding the shortest path but comes with the drawback of potentially high memory consumption, especially in large graphs. Moreover, the experiment likely highlighted BFS's suitability for small to medium-sized problems with limited branching factors. However, in scenarios with high branching factors or infinite-depth graphs, BFS may not be the most efficient choice due to its exhaustive nature. Overall, by implementing BFS, we gained a deeper understanding of its mechanics and trade-offs, empowering us to make informed decisions when selecting search algorithms for various problem domains.

Conclusion

In conclusion, the implementation of the Breadth-First Search (BFS) algorithm in this lab experiment has proven to be a valuable tool for exploring and navigating through complex problem spaces. Through its systematic exploration of neighboring nodes, BFS efficiently finds the shortest path to the goal state. This experiment has provided insight into the practical application of uninformed search algorithms, highlighting BFS's effectiveness in solving a variety of problems, from pathfinding in mazes to network routing. By understanding the principles and mechanics of BFS, we can apply this knowledge to tackle real-world problems across various domains. Moving forward, further experimentation and exploration of BFS, along with other search algorithms, will contribute to advancing our understanding of artificial intelligence and its applications in problem-solving and decision-making processes.

Viva questions

1. Can you explain the fundamental principle behind Breadth-first Search (BFS) algorithm and how it is applied in finding solutions in various problem spaces?
2. What are the key characteristics that distinguish Breadth-first Search from other uninformed search algorithms such as Depth-first Search or Iterative Deepening Search?
3. During the implementation of BFS, how do you ensure that the algorithm explores nodes level by level in a systematic manner? Can you describe the data structures or techniques utilized for this purpose?
4. What are the advantages and limitations of Breadth-first Search in terms of its time complexity, space complexity, and ability to find optimal solutions?
5. In practical applications, how would you modify or enhance the basic Breadth-first Search algorithm to address specific challenges or improve its performance in different problem domains?

Assignment / Homework

- Explain the fundamental principle behind Breadth-first Search (BFS) algorithm and how it is applied in finding solutions in various problem spaces?

Lab Report: To learn the concepts of Uninformed Search Algorithms - Breadth-first Search.

Experiment No. 07:

Experiment Name / Title: Implementation of Informed Search Algorithms - Best-first Search (Greedy search)

Aim or Objectives

To learn the concepts of informed Search Algorithms - Best-first Search (Greedy search)

Apparatus Required

- High-performance CPUs and GPUs.

Theory

Best-first search, a type of informed search algorithm, prioritizes nodes for exploration based on a heuristic function. In the context of greedy search, this heuristic guides the algorithm to select the most promising path towards the goal state. Unlike uninformed search algorithms, such as breadth-first or depth-first search, which explore all possible paths uniformly, greedy search focuses on the most promising options. It evaluates nodes using a heuristic estimation of the cost from the current node to the goal, often resulting in faster search times but with the risk of overlooking optimal solutions due to its myopic nature. The implementation of best-first search involves maintaining a priority queue where nodes are ordered based on their heuristic values, with the most promising nodes explored first. By efficiently navigating the search space, greedy search offers a practical solution for problems where optimality is less critical compared to finding a solution quickly.

Algorithm

1. **Initialize**: Start with the open list containing the initial node.
2. **Expand Node**: Dequeue the node with the lowest heuristic value h (best estimate to goal).
3. **Check Goal**: Return the path if the goal node is reached.
4. **Generate Children**: Add child nodes to the open list with their heuristic values.
5. **Update List**: Maintain the open list ordered by heuristic values.
6. **Repeat**: Continue until the goal is found or the list is empty.

Code or Script

```
graph = {
    'A': [('B', 3), ('C', 5)],
    'B': [('D', 2)],
    'C': [('E', 4)],
    'D': [('F', 1)],
    'E': [],
    'F': []
}
sn = 'A'
```

```
gn = 'F'

def gbfs(graph, s, g, h):
    # Custom priority queue to store nodes based on heuristic value
    pq = [(h(s, g), s)]
    visited = set()
    while pq:
        # Sort the priority queue based on heuristic values
        pq.sort()
        # Pop the node with the lowest heuristic value
        current_cost, current_node = pq.pop(0)
        # Check if the goal is reached
        if current_node == g:
            print("Goal reached:", current_node)
            return
        # Explore neighbors of the current node
        for neighbor, cost in graph[current_node]:
            if neighbor not in visited:
                # Calculate heuristic value for the neighbor
                neighbor_cost = h(neighbor, g)
                pq.append((neighbor_cost, neighbor))
                visited.add(neighbor)
        print("Visited node:", current_node)
    print("Goal not reached")
# A simple heuristic function (Euclidean distance in this case)
def euclidean_distance(n, g):
    return abs(ord(n) - ord(g))



gbfs(graph, sn, gn, euclidean_distance)
```

Outcome/ Results (input, output etc.)

```
Visited node: A
Visited node: C
Visited node: E
```

```
Visited node: B
Visited node: D
Goal reached: F
```

Discussion

In the implementation of Best-first Search (Greedy search) in this lab experiment, several key insights and observations arose. Firstly, the algorithm's greedy nature, prioritizing nodes based solely on heuristic information, was evident in its behavior. This resulted in rapid exploration towards promising solutions, often leading to quick convergence. However, this approach lacked the foresight to backtrack in case of suboptimal decisions, occasionally leading to premature termination or reaching local optima. Furthermore, the experiment shed light on the significance of heuristic functions in guiding the search process. Variations in heuristic functions significantly impacted the algorithm's performance, influencing both the efficiency and quality of solutions found. Consequently, the choice of heuristic function proved crucial in striking a balance between exploration and exploitation. Moreover, the trade-off between optimality and efficiency became apparent, as the algorithm's greedy nature favored quick solutions at the expense of potentially sacrificing optimality. This trade-off prompted discussions on the application domains where such an approach is suitable, emphasizing scenarios where quick, near-optimal solutions are preferred over exhaustive search for the global optimum. Overall, the lab experiment provided valuable insights into the workings of Best-first Search (Greedy search), highlighting its strengths, limitations, and factors influencing its performance. These insights pave the way for further exploration and refinement of informed search algorithms in various problem domains.

Conclusion

In conclusion, the implementation of the Best-first Search, also known as Greedy search, in this lab experiment has demonstrated its efficacy in navigating search spaces efficiently. By prioritizing nodes based on heuristic evaluation, Greedy search swiftly narrows down the search towards promising solutions, often leading to quick convergence. However, its greedy nature may overlook optimal paths, resulting in suboptimal solutions. Despite this limitation, the algorithm's simplicity and computational efficiency make it a valuable tool for various applications, particularly in domains where real-time decision-making is crucial. Additionally, this experiment has underscored the significance of heuristic functions in guiding search

algorithms effectively. Future exploration could involve refining heuristics to strike a better balance between exploration and exploitation, thereby enhancing the performance of Greedy search in solving complex problems. Overall, this experiment provides valuable insights into the practical implications and trade-offs associated with employing informed search algorithms like Best-first Search.

Viva questions

1. How does the Best-first Search (Greedy search) algorithm differ from other informed search algorithms, such as A* search?
2. Can you explain the heuristic function used in the Best-first Search algorithm and its role in guiding the search process?
3. What are the advantages and disadvantages of using the Best-first Search algorithm in comparison to uninformed search algorithms like Breadth-first Search or Depth-first Search?
4. In what scenarios would the Best-first Search algorithm outperform other search strategies, and conversely, when might it be less effective?
5. How does the choice of heuristic function impact the efficiency and effectiveness of the Best-first Search algorithm, particularly in terms of finding optimal solutions versus quickly finding satisfactory solutions?

Assignment / Homework

- What scenarios would the Best-first Search algorithm outperform other search strategies, and conversely, when might it be less effective?

Lab Report: To learn the concepts of informed Search Algorithms - Best-first Search (Greedy search)

Experiment No. 08:

Experiment Name / Title: Implementation of Informed Search Algorithms - A* Search Algorithm

Aim or Objectives

To learn the concepts of informed Search Algorithms - A* Search

Apparatus Required

- High-performance CPUs and GPUs.

Theory

A* (pronounced "A-star") is a powerful graph traversal and pathfinding algorithm widely used in artificial intelligence and computer science. It is mainly used to find the shortest path between two nodes in a graph, given the estimated cost of getting from the current node to the destination node. The main advantage of the algorithm is its ability to provide an optimal path by exploring the graph in a more informed way compared to traditional search algorithms such as Dijkstra's algorithm.

Algorithm A* combines the advantages of two other search algorithms: Dijkstra's algorithm and Greedy Best-First Search. Like Dijkstra's algorithm, A* ensures that the path found is as short as possible but does so more efficiently by directing its search through a heuristic similar to Greedy Best-First Search. A heuristic function, denoted h(n), estimates the cost of getting from any given node n to the destination node.

The main idea of A* is to evaluate each node based on two parameters:

g(n): the actual cost to get from the initial node to node n. It represents the sum of the costs of node n outgoing edges.
h(n): Heuristic cost (also known as "estimation cost") from node n to destination node n. This problem-specific heuristic function must be acceptable, meaning it never overestimates the actual cost of achieving the goal. The evaluation function of node n is defined as f(n) = g(n) h(n).
Algorithm A* selects the nodes to be explored based on the lowest value of f(n), preferring the nodes with the lowest estimated total cost to reach the goal. The A* algorithm works:

- Create an open list of found but not explored nodes.
- Create a closed list to hold already explored nodes.
- Add a starting node to the open list with an initial value of g

- Repeat the following steps until the open list is empty or you reach the target node:
- Find the node with the smallest f-value (i.e., the node with the minor g(n) h(n)) in the open list.
- Move the selected node from the open list to the closed list.
- Create all valid descendants of the selected node.
- For each successor, calculate its g-value as the sum of the current node's g value and the cost of moving from the current node to the successor node. Update the g-value of the tracker when a better path is found.
- If the follower is not in the open list, add it with the calculated g-value and calculate its h-value. If it is already in the open list, update its g value if the new path is better.
- Repeat the cycle. Algorithm A* terminates when the target node is reached or when the open list empties, indicating no paths from the start node to the target node. The A* search algorithm is widely used in various fields such as robotics, video games, network routing, and design problems because it is efficient and can find optimal paths in graphs or networks.

Algorithm

1. Start with the open list containing the start node.
2. Dequeue the node with the lowest `f = g + h` value.
3. Return if the goal node is reached.
4. Enqueue child nodes with updated `g` and `h` values.
5. Track and update `g` and `h` for nodes.
6. Skip nodes with higher costs if a better path is found.

Code or Script

```
import heapq

def a_star_search(graph, start, goal, heuristic):
    # Priority queue to store (cost, current_node, path)
    open_set = []
    heapq.heappush(open_set, (0, start, [start]))

    # Dictionary to store the cost of the shortest path to each
node
    g_costs = {start: 0}
```

```
    while open_set:
        current_cost,        current_node,        path        =
heapq.heappop(open_set)

        # If we reach the goal, return the path
        if current_node == goal:
            return path

        # Explore neighbors
        for neighbor, cost in graph[current_node].items():
            tentative_g_cost = g_costs[current_node] + cost

            # If the new path is cheaper, or the neighbor hasn't
been explored yet
            if neighbor not in g_costs or tentative_g_cost <
g_costs[neighbor]:
                g_costs[neighbor] = tentative_g_cost
                f_cost = tentative_g_cost + heuristic[neighbor]
                heapq.heappush(open_set,   (f_cost,   neighbor,
path + [neighbor]))

    return None  # If there's no path to the goal

# Example usage:
graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'A': 1, 'C': 2, 'D': 5},
    'C': {'A': 4, 'B': 2, 'D': 1},
    'D': {'B': 5, 'C': 1, 'E': 3},
    'E': {'D': 3}
}

heuristic = {
```

```
    'A': 7,
    'B': 6,
    'C': 2,
    'D': 1,
    'E': 0  # Goal node heuristic is always 0
}

start = 'A'
goal = 'E'
path = a_star_search(graph, start, goal, heuristic)

print(f"Path from {start} to {goal}: {path}")
```

Outcome/ Results (input, output etc.)

```
Path from A to E: ['A', 'B', 'C', 'D', 'E']
```

Discussion

In this lab experiment, we implemented the A* search algorithm to find the shortest path in a weighted graph. The A* algorithm is a widely used path finding and graph traversal algorithm due to its efficiency and optimal, making it suitable for various applications such as routing, game development, and robotics.

Implementation Details

The graph was represented using an adjacency list, where each node is connected to its neighbors with associated costs. The heuristic function, which is essential for the A* algorithm, provides an estimated cost from each node to the goal node. In this experiment, the heuristic values were manually defined to reflect the estimated distance to the goal.

Algorithm Performance

The A* algorithm was successful in finding the shortest path from the start node 'A' to the goal node 'E'. The algorithm uses a priority queue to explore the nodes in the order of their estimated total cost (the sum of the actual cost from the start node to the current node and the heuristic cost from the current node to the goal). This approach ensures that the most promising nodes are explored first, leading to an efficient search process.

The resulting path ['A', 'B', 'C', 'D', 'E'] demonstrates the algorithm's ability to correctly evaluate and prioritize nodes, ensuring that the path found is indeed the shortest. The total cost of this path matches the expected optimal cost, confirming the correctness of the implementation.

Conclusion

This experiment successfully demonstrated the implementation and effectiveness of the A* search algorithm in finding the shortest path in a weighted graph. The algorithm efficiently navigated through the graph using a priority queue and a heuristic function, resulting in the optimal path from the start to the goal node. The results validate the theoretical properties of the A* algorithm, including its completeness and optimal, given an admissible heuristic.

Future work could explore more complex graphs, different heuristic functions, and performance optimization techniques. Additionally, applying the A* algorithm to real-world problems can provide further insights into its practical applications and limitations. This experiment underscores the importance of algorithmic design and heuristic selection in solving path finding problems efficiently.

Viva questions

1. Can you explain the role of the heuristic function in the A* search algorithm and why it is critical for the algorithm's performance?

2. How does the A* algorithm ensure that the path it finds is the shortest possible path in the graph?

3. What are the potential challenges and limitations of using the A* search algorithm in large-scale or dynamic environments?

Assignment / Homework

To deepen your understanding of the A* search algorithm through implementation, experimentation, and analysis. This assignment will help you explore various aspects of the algorithm, including heuristic functions, performance on different graph structures, and practical applications.

Tasks:

Basic Implementation:

Implement the A* search algorithm in Python as demonstrated in the lab experiment.

Ensure your implementation can handle different graphs and heuristic functions.

Lab Report: To learn the concepts of informed Search Algorithms - A* Search

Experiment No. 09:

Experiment Name / Title: Implementation of Adversarial Search - Mini max Algorithm

Aim or Objectives

To learn the concepts of Adversarial Search - Mini max Algorithm

Apparatus Required

- High-performance CPUs and GPUs.

Theory

1. Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory. It provides an optimal move for the player assuming that opponent is also playing optimally.
2. Mini-Max algorithm uses recursion to search through the game-tree.
3. Min-Max algorithm is mostly used for game playing in AI. Such as Chess, Checkers, tic-tac-toe, go, and various tow-players game. This Algorithm computes the minimax decision for the current state.
4. In this algorithm two players play the game, one is called MAX and other is called MIN.
5. Both the players fight it as the opponent player gets the minimum benefit while they get the maximum benefit.
6. Both Players of the game are opponent of each other, where MAX will select the maximized value and MIN will select the minimized value.
7. The minimax algorithm performs a depth-first search algorithm for the exploration of the complete game tree.
8. The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.

Algorithm

1. Recursive function for evaluating game states.
2. Return score if depth is 0 or game is terminal.
3. Choose the highest score from child nodes.
4. Choose the lowest score from child nodes.
5. Use `get_children` to simulate possible moves.
6. Score terminal states with `evaluate` function.

Code or Script

```
import math

class GameState:
    def __init__(self, player):
        self.board = [[' ' for _ in range(3)] for _ in range(3)]
        self.player = player
```

```
    def display(self):
        for row in self.board:
            print('|'.join(row))
        print()

    def available_moves(self):
        moves = []
        for i in range(3):
            for j in range(3):
                if self.board[i][j] == ' ':
                    moves.append((i, j))
        return moves

    def make_move(self, move):
        i, j = move
        self.board[i][j] = self.player
        self.player = 'X' if self.player == 'O' else 'O'

    def undo_move(self, move):
        i, j = move
        self.board[i][j] = ' '
        self.player = 'X' if self.player == 'O' else 'O'

    def is_winner(self, player):
        for i in range(3):
            if all(self.board[i][j] == player for j in
range(3)):
                return True
            if all(self.board[j][i] == player for j in
range(3)):
                return True
        if all(self.board[i][i] == player for i in range(3)):
            return True
```

```
        if all(self.board[i][2 - i] == player for i in range(3)):
            return True
        return False

    def game_over(self):
        return self.is_winner('X') or self.is_winner('O') or not any(' ' in row for row in self.board)

def minimax(state, depth, maximizing_player):
    if state.game_over() or depth == 0:
        if state.is_winner('X'):
            return -1
        elif state.is_winner('O'):
            return 1
        else:
            return 0

    if maximizing_player:
        max_eval = -math.inf
        for move in state.available_moves():
            state.make_move(move)
            eval = minimax(state, depth - 1, False)
            state.undo_move(move)
            max_eval = max(max_eval, eval)
        return max_eval
    else:
        min_eval = math.inf
        for move in state.available_moves():
            state.make_move(move)
            eval = minimax(state, depth - 1, True)
            state.undo_move(move)
            min_eval = min(min_eval, eval)
        return min_eval
```

```
def best_move(state):
    best_score = -math.inf
    best_move = None
    for move in state.available_moves():
        state.make_move(move)
        score = minimax(state, 3, False)
        state.undo_move(move)
        if score > best_score:
            best_score = score
            best_move = move
    return best_move

if __name__ == "__main__":
    game = GameState('X')
    game.display()
    while not game.game_over():
        if game.player == 'X':
            move = best_move(game)
            game.make_move(move)
            print("X makes move:", move)
        else:
            move = tuple(map(int, input("Enter your move (row
col): ").split()))
            game.make_move(move)
        game.display()
    if game.is_winner('X'):
        print("X wins!")
    elif game.is_winner('O'):
        print("O wins!")
    else:
        print("It's a draw!")
```

Outcome/ Results (input, output etc.)

```
 | |
 | |
 | |

X makes move: (0, 0)
X| |
 | |
 | |

Enter your move (row col): 1 1
X| |
 |O|
 | |

X makes move: (0, 1)
X|X|
 |O|
 | |

Enter your move (row col): 2 0
X|X|
 |O|
O| |

X makes move: (1, 0)
X|X|
X|O|
O| |

Enter your move (row col): 2 1
X|X|
X|O|
O|O|
```

```
X makes move: (2, 2)
X|X|
X|O|
O|O|X

X wins!
```

Discussion

In this lab experiment, The working of the minimax algorithm can be easily described using an example. Below we have taken an example of game-tree which is representing the two-player game.

In this example, there are two players one is called Maximizer and other is called Minimizer.

Maximizer will try to get the Maximum possible score, and Minimizer will try to get the minimum possible score.

This algorithm applies DFS, so in this game-tree, we have to go all the way through the leaves to reach the terminal nodes.

At the terminal node, the terminal values are given so we will compare those value and backtrack the tree until the initial state occurs.That was the complete workflow of the minimax two player game.

Conclusion

In conclusion, the provided Python implementation offers a simplified demonstration of the minimax algorithm applied to the classic game of Tic-Tac-Toe. Through this implementation, several key concepts have been highlighted:

Firstly, the `GameState` class effectively represents the state of the game, encapsulating essential functionalities such as move generation, outcome evaluation, and player alternation. This clear abstraction of the game state facilitates the application of the minimax algorithm.

Secondly, the minimax algorithm itself serves as a powerful decision-making tool, recursively traversing the game tree to evaluate potential moves and select the optimal one. By considering the game from both the maximizing player's and the minimizing player's perspectives, the algorithm ensures strategic decision-making that aims to either maximize the AI's chances of winning or minimize the opponent's.

Furthermore, the discussion around the depth parameter underscores the trade-off between computational complexity and decision quality. By limiting the depth of the search, the algorithm strikes a balance between efficiency and effectiveness, enabling practical implementations in real-time scenarios.

Moreover, while the presented implementation is tailored to Tic-Tac-Toe, the underlying principles of the minimax algorithm are applicable to a wide range of turn-based games and decision-making problems. Understanding these principles lays the groundwork for exploring more advanced techniques, such as alpha-beta pruning, to enhance the algorithm's efficiency further.

Overall, this implementation provides a foundational understanding of the minimax algorithm and its application in game AI. Through its simplicity and clarity, it serves as a valuable starting point for exploring more complex game scenarios and algorithmic optimizations in the pursuit of creating intelligent game-playing agents.

Viva questions

1. How does the minimax algorithm work?: Explain the basic principles of the minimax algorithm and how it's applied to decision-making in games.

2. What is the significance of the depth parameter in the minimax function?: Discuss how the depth parameter affects the performance and computational complexity of the minimax algorithm.

3. How does the game state representation facilitate the implementation of the minimax algorithm?: Explain the role of the GameState class and its methods in representing the game state and enabling move generation and evaluation.

Lab Report: To learn the concepts of Adversarial Search - Mini max Algorithm

Experiment No. 10:

Experiment Name / Title: Implementation of Adversarial Search - Alpha Beta Algorithm

Aim or Objectives

To learn the concepts of Adversarial Search - Alpha Beta Algorithm

Apparatus Required

- High-performance CPUs and GPUs.

Theory

Alpha-Beta pruning is not actually a new algorithm, but rather an optimization technique for the minimax algorithm. It reduces the computation time by a huge factor. This allows us to search much faster and even go into deeper levels in the game tree. It cuts off branches in the game tree which need not be searched because there already exists a better move available. It is called Alpha-Beta pruning because it passes 2 extra parameters in the minimax function, namely alpha and beta.

Let's define the parameters alpha and beta.

Alpha is the best value that the maximizer currently can guarantee at that level or above.

Beta is the best value that the minimizer currently can guarantee at that level or below.

Algorithm

1. Extend `minimax` with alpha and beta values.
2. Return score if depth is 0 or game is terminal.
3. Update alpha and prune branches if score exceeds beta.
4. Update beta and prune branches if score is less than alpha.
5. Cut off branches where further exploration isn't needed.
6. Use `get_children` to explore possible moves.

Code or Pseudocode

```
# Initial values of Alpha and Beta
MAX, MIN = 1000, -1000

# Returns optimal value for current player
#(Initially called for root and maximizer)
def minimax(depth, nodeIndex, maximizingPlayer,
```

```
          values, alpha, beta):

# Terminating condition. i.e
# leaf node is reached
if depth == 3:
     return values[nodeIndex]

if maximizingPlayer:

     best = MIN

     # Recur for left and right children
     for i in range(0, 2):

          val = minimax(depth + 1, nodeIndex * 2 + i,
                         False, values, alpha, beta)
          best = max(best, val)
          alpha = max(alpha, best)

          # Alpha Beta Pruning
          if beta <= alpha:
               break

     return best

else:
     best = MAX

     # Recur for left and
     # right children
     for i in range(0, 2):

          val = minimax(depth + 1, nodeIndex * 2 + i,
                                True, values, alpha, beta)
```

```
                best = min(best, val)
                beta = min(beta, best)

                # Alpha Beta Pruning
                if beta <= alpha:
                     break

          return best

# Driver Code
if __name__ == "__main__":

     values = [3, 5, 6, 9, 1, 2, 0, -1]
     print("The optimal value is :", minimax(0, 0, True, values,
MIN, MAX))
```

Outcome/ Results (input, output etc.)

The optimal value is : 5

Discussion

The Alpha-Beta algorithm is a fundamental search algorithm in the field of artificial intelligence (AI), primarily used in game theory and for decision-making processes in two-player games like chess, checkers, and tic-tac-toe. It is an optimization of the minimax algorithm, which is a recursive algorithm used to choose an optimal move for a player assuming that the opponent is also playing optimally. The primary purpose of the Alpha-Beta algorithm is to reduce the number of nodes evaluated in the search tree by the minimax algorithm, thereby increasing efficiency without affecting the outcome.

Conclusion

The Alpha-Beta algorithm traverses the game tree in a depth-first manner. It prunes (cuts off) branches that cannot possibly influence the final decision, using the Alpha and Beta values to guide the pruning process.

In summary, the Alpha-Beta algorithm is a crucial optimization technique in AI for adversarial search problems, enabling more efficient computation without compromising the accuracy of the optimal decision-making process.

Viva questions

1. How does move ordering affect the performance of the Alpha-Beta algorithm?
2. What are the best-case and worst-case scenarios for the Alpha-Beta algorithm in terms of pruning?
3. Explain the significance of the terminal node in the Alpha-Beta algorithm.

Lab Report: To learn the concepts of Adversarial Search - Alpha Beta Algorithm

Experiment No. 11:

Experiment Name / Title: Implementation of Uninformed Search - Depth-limited Search Algorithm

Aim or Objectives

To learn the concepts of Uninformed Search - Depth-limited Search Algorithm

Apparatus Required

- High-performance CPUs and GPUs.

Theory

Depth-Limited Search (DLS) is a variant of the Depth-First Search (DFS) algorithm used in artificial intelligence and computer science for traversing or searching tree or graph data structures. Unlike traditional DFS, which can potentially explore an entire infinite search space, DLS imposes a limit on the depth of exploration, thus avoiding the pitfalls of infinite loops or excessively deep searches.

A depth-limited search algorithm is similar to depth-first search with a predetermined limit. Depth-limited search can solve the drawback of the infinite path in the Depth-first search. In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

Depth-limited search can be terminated with two Conditions of failure:

Standard failure value: It indicates that problem does not have any solution.

Cutoff failure value: It defines no solution for the problem within a given depth limit.

Algorithm

1. Recursive function with a depth limit.
2. Return node value if depth is 0 or goal is reached.
3. Explore children if depth > 0.
4. Stop if depth limit is reached.
5. Use `get_children` to simulate moves.
6. Aggregate results from child nodes.

Code

Consider the following simple tree:

A

```
     / \
    B   C
   / \   / \
  D   E F   G
```

Here, A is the root node, and we are searching for the goal node "G" with a depth limit L = 2.

```
def depth_limited_search(node, goal, limit):
    return recursive_dls(node, goal, limit)

def recursive_dls(node, goal, limit):
    if node == goal:
        return node
    elif limit == 0:
        return "cutoff"
    else:
        cutoff_occurred = False
        for child in expand(node):
            result = recursive_dls(child, goal, limit - 1)
            if result == "cutoff":
                cutoff_occurred = True
            elif result is not None:
                return result
        return "cutoff" if cutoff_occurred else None

def expand(node):
    tree = {
        'A': ['B', 'C'],
        'B': ['D', 'E'],
        'C': ['F', 'G'],
        'D': [], 'E': [], 'F': [], 'G': []
    }
    return tree[node]
```

```
# Example usage:
root = 'A'
goal = 'G'
depth_limit = 2
result = depth_limited_search(root, goal, depth_limit)
print(result)
```

Outcome/ Results (input, output etc.)

G

Discussion

Depth-Limited Search (DLS) is a useful modification of the Depth-First Search (DFS) algorithm, designed to handle the limitations and challenges posed by unbounded search spaces. By introducing a depth limit, DLS can effectively manage resources and prevent infinite loops, making it a valuable tool in various applications of artificial intelligence and computer science. Here are some key points:

- Completeness and Optimality
- Completeness
- Practical Applications
- Performance
- Space Complexity
- Time Complexity

Conclusion

Depth-Limited Search is a strategic enhancement of the Depth-First Search algorithm, offering a balance between resource management and search thoroughness. By limiting the depth of exploration, DLS can efficiently handle large or infinite search spaces, making it applicable in real-time systems, game playing, and AI problem-solving. However, its effectiveness is closely tied to the chosen depth limit, which can be a critical factor in determining the success of the search.

While DLS is not complete or optimal on its own, it serves as a foundational component for more advanced search techniques like Iterative Deepening Depth-First Search, which iteratively applies DLS with increasing limits to ensure completeness and optimality.

Understanding the strengths and limitations of DLS is essential for its effective application in various domains of artificial intelligence and computer science.

Viva questions

1. In what situations is DLS more advantageous compared to unbounded DFS or BFS?
2. How would you choose an appropriate depth limit L for a DLS?
3. What are the potential drawbacks of choosing a depth limit that is too low or too high?

Lab Report: To learn the concepts of Uninformed Search - Depth-limited Search Algorithm

Experiment No. 12:

Experiment Name / Title: Implementation of Uninformed Search -Uniform cost search Algorithm

Aim or Objectives

To learn the concepts of Uninformed Search - Uniform cost search Algorithm

Apparatus Required

- High-performance CPUs and GPUs.

Theory

Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph. This algorithm comes into play when a different cost is available for each edge. The primary goal of the uniform-cost search is to find a path to the goal node which has the lowest cumulative cost. Uniform-cost search expands nodes according to their path costs form the root node. It can be used to solve any graph/tree where the optimal cost is in demand. A uniform-cost search algorithm is implemented by the priority queue. It gives maximum priority to the lowest cumulative cost. Uniform cost search is equivalent to BFS algorithm if the path cost of all edges is the same.

Advantages:

Uniform cost search is optimal because at every state the path with the least cost is chosen.

Disadvantages:

It does not care about the number of steps involve in searching and only concerned about path cost. Due to which this algorithm may be stuck in an infinite loop.

Algorithm

1. Start with the root node and cost 0.
2. Dequeue the node with the lowest cost.
3. Return if the goal node is reached.
4. Enqueue child nodes with updated costs.
5. Ensure lowest cost paths are explored.
6. Continue until the queue is empty or goal is found.

Code

```
import heapq
```

```
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, start, end, cost):
        if start not in self.graph:
            self.graph[start] = []
        self.graph[start].append((end, cost))

    def uniform_cost_search(self, start, goal):
        explored = set()
        frontier = [(0, start, [])]  # (cost, node, path)

        while frontier:
            cost, node, path = heapq.heappop(frontier)
            if node in explored:
                continue
            path = path + [node]
            if node == goal:
                return path
            explored.add(node)
            for neighbor, new_cost in self.graph.get(node, []):
                if neighbor not in explored:
                    heapq.heappush(frontier, (cost + new_cost,
neighbor, path))

        return None

# Example usage
graph = Graph()
graph.add_edge('A', 'B', 4)
graph.add_edge('A', 'C', 2)
graph.add_edge('B', 'C', 5)
graph.add_edge('B', 'D', 10)
```

```
graph.add_edge('C', 'D', 3)
graph.add_edge('C', 'E', 8)
graph.add_edge('D', 'E', 6)

start_node = 'A'
goal_node = 'E'
path = graph.uniform_cost_search(start_node, goal_node)
if path:
    print("Path from", start_node, "to", goal_node, ":", path)
else:
    print("No path found from", start_node, "to", goal_node)
```

Outcome/ Results (input, output etc.)

Path from A to E : ['A', 'C', 'D', 'E']

Discussion

Uniform Cost Search (UCS) is a search algorithm used in artificial intelligence and computer science to find the least-cost path from a start node to a goal node in a weighted graph. UCS is a variant of Dijkstra's algorithm, and it guarantees finding the optimal solution by exploring the lowest cumulative cost paths first.\

Key Characteristics

Priority Queue: UCS uses a priority queue (often implemented with a min-heap) to keep track of nodes to be explored. The priority of each node is determined by the cumulative cost from the start node.

Explores Paths with Lowest Cost First: Unlike Depth-First Search (DFS) or Breadth-First Search (BFS), UCS expands the node with the lowest path cost, ensuring that the first time a goal node is reached, it is reached via the least-cost path.

Optimality: UCS is optimal, meaning it guarantees finding the lowest-cost solution if the cost of each step is positive.

Completeness: UCS is complete, provided that the search space is finite and the step costs are non-negative.

Conclusion

Uniform Cost Search is a powerful and reliable algorithm for finding the optimal path in a weighted graph, particularly useful when path costs vary. It guarantees both completeness and

optimality, making it a valuable tool in many practical applications, such as route planning, robotics, and network routing.

However, UCS's significant memory and time requirements can limit its practicality in very large search spaces. These constraints necessitate considering trade-offs between the quality of the solution and available computational resources. In scenarios where path cost is critical and resources permit, UCS remains an excellent choice for finding the least-cost path. For large or complex search spaces, enhancements like A* search, which combines UCS with heuristics to guide the search more efficiently, can be more appropriate.

Viva questions

1. What is the time complexity of UCS?
2. What is the space complexity of UCS?
3. Is UCS a complete search algorithm? Explain.
4. Is UCS optimal? Why or why not?

Lab Report: To learn the concepts of Uninformed Search -Uniform Cost Search Algorithm.

Experiment No. 13:

Experiment Name / Title: Implement 8-Queens Problem using Python.

Aim or Objectives

To learn the concepts of Implement 8-Queens Problem using Python.

Apparatus Required

- High-performance CPUs and GPUs.

Theory

The eight queens problem is the problem of placing eight queens on an 8×8 chessboard such that none of them attack one another (no two are in the same row, column, or diagonal). More generally, the n queens problem places n queens on an n×n chessboard. There are different solutions for the problem.

Explanation:

- This **pseudocode uses a backtracking algorithm** to find a solution to the 8 Queen problem, which consists of placing 8 queens on a chessboard in such a way that no two queens threaten each other.
- The algorithm starts by placing a queen on the first column, then it proceeds to the next column and places a queen in the **first safe** row of that column.
- If the algorithm reaches the 8th column and al**l queens are placed in a safe position**, it prints the board and returns true.
  If the algorithm is unable to place a queen in a safe position in a certain column, it backtracks to the previous column and tries a different row.
- The "isSafe" function **checks** if it is safe to place a queen on a certain row and column by checking if there are any queens in the same row, diagonal or anti-diagonal.
- It's worth to notice that this is just a high-level **pseudocode** and it might need to be adapted depending on the specific implementation and language you are using.

Algorithm

1. Recursive function to place queens on the board.
2. Return a solution when all 8 queens are placed.

3. Ensure no two queens threaten each other.
4. Try placing a queen in each column of the current row.
5. Remove and re-position queens if conflicts arise.
6. Collect and return valid board configurations.

Code

```
N = 8 # (size of the chessboard)

def solveNQueens(board, col):
     if col == N:
          print(board)
          return True
     for i in range(N):
          if isSafe(board, i, col):
               board[i][col] = 1
               if solveNQueens(board, col + 1):
                    return True
               board[i][col] = 0
     return False

def isSafe(board, row, col):
     for x in range(col):
          if board[row][x] == 1:
               return False
     for x, y in zip(range(row, -1, -1), range(col, -1, -1)):
          if board[x][y] == 1:
               return False
     for x, y in zip(range(row, N, 1), range(col, -1, -1)):
          if board[x][y] == 1:
               return False
     return True

board = [[0 for x in range(N)] for y in range(N)]
if not solveNQueens(board, 0):
```

```
    print("No solution found")
```

Outcome/ Results (input, output etc.)

```
[[1, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 1], [0, 1, 0, 0, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 0, 0, 1, 0, 0], [0, 0, 1, 0, 0, 0, 0, 0]]
```

Discussion

Time Complexity : O((m + q) log^2 n)

Space Complexity : O((m + q) log n)

Implementing the 8-Queens Problem in Python involves placing eight queens on a chessboard so that no two queens threaten each other. This classic problem can be solved using backtracking, where the algorithm incrementally places queens on the board and backtracks whenever a conflict occurs. The Python implementation typically uses recursion to explore all possible placements, checking for conflicts in rows, columns, and diagonals. Efficient pruning of the search space ensures optimal performance. The simplicity of Python's data structures, like lists and sets, makes it an ideal language for implementing and visualizing the solution.

Conclusion

Implementing the 8-Queens Problem using Python showcases the power of algorithmic problem-solving and computational efficiency. By leveraging techniques such as backtracking, developers can explore numerous potential solutions while effectively pruning viable options, demonstrating Python's versatility and ease of use in handling combinatorial challenges. This classic problem not only reinforces fundamental programming concepts but also highlights the importance of optimization in algorithm design, providing a robust framework for tackling more complex issues in artificial intelligence and operations research. Overall, the implementation serves as a practical exercise in enhancing logical thinking and coding proficiency.

Viva questions

1. Explain the backtracking algorithm used in your implementation of the 8-Queens Problem. How does the algorithm ensure that no two queens threaten each other on the board?
2. Describe how you represent the chessboard in your Python code. How do you check for conflicts when placing a new queen on the board?

Lab Report: To learn the concepts of Implement 8-Queens Problem using Python.

Complex Engineering Problems:

Problem 1: Advanced Pathfinding with Informed Search Algorithms

Background:

You are developing an AI for an autonomous drone that needs to navigate through a complex urban environment. The environment is represented as a grid where each cell can either be open, contain an obstacle, or be a part of a designated no-fly zone. The drone must find the shortest path from its starting position to its destination while avoiding obstacles and no-fly zones.

Requirements:

1. Grid Representation:
   - Implement the grid using Python dictionaries and lists. The grid should be a 2D array where each cell is either 0 (open), 1 (obstacle), or 2 (no-fly zone).

2. Search Algorithms:
   - Implement the Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms to find paths from the start to the goal.
   - Implement the A* search algorithm and compare its performance with BFS and DFS.

3. Heuristics for A*:
   - Design an admissible heuristic for the A* algorithm based on the Manhattan distance.

4. Performance Analysis:
   - Compare the performance of BFS, DFS, and A* in terms of time complexity, space complexity, and path optimality.

5. Fuzzy Logic for Path Adjustment:
   - Implement a fuzzy logic system to adjust the drone's path in real-time to avoid dynamic obstacles (e.g., moving objects) that appear after the initial path has been planned.

Deliverables:

- Python code for the grid representation and all search algorithms.
- A report comparing the performance of BFS, DFS, and A*.
- Documentation of the fuzzy logic system and how it integrates with the A* pathfinding.
- A presentation demonstrating the pathfinding and real-time path adjustment using a visual representation of the grid.

Problem 2: Adversarial Search in Real-Time Strategy Games

Background:
You are tasked with developing an AI for a real-time strategy (RTS) game. The game involves two players, each controlling units on a grid. The objective is to capture the opponent's base while defending your own. The AI should make strategic decisions in real-time, considering the opponent's possible moves.

Requirements:
1. Game Representation:
   - Implement the game grid using Python lists and dictionaries. The grid should be a 2D array where each cell can be empty, contain a unit, or contain a base.

2. Adversarial Search:
   - Implement the Minimax algorithm to decide the best move for the AI, assuming the opponent is also playing optimally.
   - Implement Alpha-Beta pruning to improve the efficiency of the Minimax algorithm.

3. Heuristic Evaluation Function:
   - Design a heuristic evaluation function that considers factors such as the number of units, proximity to the opponent's base, and control of key grid areas.

4. Real-Time Decision Making:
   - Adapt the Minimax algorithm to work in real-time, allowing the AI to make decisions within a limited time frame.

5. Performance Analysis:

- Analyze the performance of the Minimax algorithm with and without Alpha-Beta pruning in terms of time complexity and decision quality.

6. Project Proposal:

- Propose a real-world problem that can be solved using adversarial search techniques. This could involve strategic planning in autonomous systems, competitive robotics, or other relevant applications.

Deliverables:

- Python code for the game representation, Minimax algorithm, and Alpha-Beta pruning.
- A report analyzing the performance and effectiveness of the algorithms.
- Documentation of the heuristic evaluation function and its impact on AI decision making.
- A project proposal for a real-world application of adversarial search techniques.
- A presentation demonstrating the AI's decision-making process in the RTS game, including real-time gameplay footage.